Paper Review • arXiv:2610.42096 • 2026-10-30

CausalWAM

Learning the Causes of Actions in World Action Models

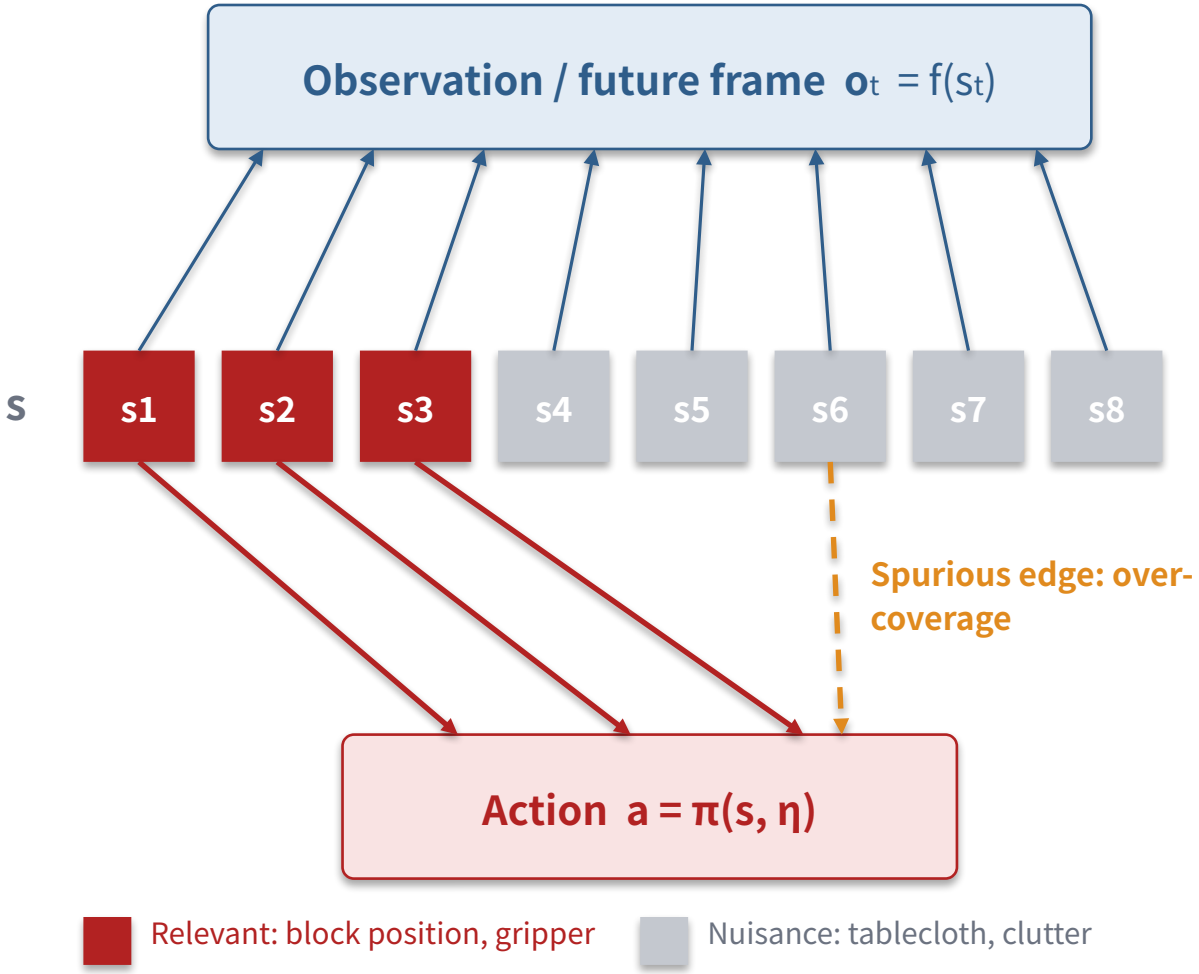
Juntao Cheng, Jinzhou Tang, Fan Feng, Henrik I. Christensen, Yujia Zheng （UIUC • UCSD）

One-sentence summary

A world action model must encode the whole scene to predict, yet actions depend on only part of it. The authors prove that imposing sparsity on the latent→action dependency identifies this action-relevant latent block from demonstrations alone. Adding a sparse routing branch to Cosmos 3 raises RoboCasa365 success from 0.24→0.43 and real-robot block stacking from 0.46→0.82.

Problem

Full scene to predict, a slice to act—demos can't split them



Why can't they be separated?

- Any invertible transform ϕ of the latents leaves the (o, a) distribution unchanged
- The transform can mix tablecloth information into the "action" variables
- A flexible enough action head still fits the demos with equally low loss

Consequence: over-coverage

The learned action mechanism depends on more latents than the true one; nuisances reach actions via "spurious edges"

Our question: from demos alone, can "what causes actions" be separated from "what prediction needs"?

Positioning

Prior identifiability fails for robot demos; only actions are constrained

Auxiliary / temporal info

Nonlinear ICA uses domain or time indices; causal processes need sufficient temporal dependence or latent-dynamics conditions

× Some demo time steps are nearly independent; latent dynamics may not be Markovian

Known-target interventions

Pin down latents via controlled interventions

× Interventions are too costly or impossible in robotics

Identify all latents

Restricted function classes / sparse dependency structure; conditions must hold for all latents

× Even variables a robot can ignore, like tablecloth or clutter, must satisfy them

This work: let actions reveal which latents matter

Sparsity is imposed only on the action side; prediction still uses all latents. No interventions, no temporal dependence, no assumptions on how hidden regimes evolve, and action-relevant and nuisance variables may be statistically correlated.

Core theory: sparsity turns “at least these” into “exactly these”

Prop. 1 Lower bound

Without sparsity

$$\| \hat{I}_i \|_0 \geqslant \| I_i \|_0 \text{ (per action dim.)}$$

Fitting demos only ensures each action dim depends on at least the true latents; spurious edges may appear

Thm. 1 Structure identification

$+ \| I(J_{\hat{\pi}}) \|_0 \leqslant \| I(J_{\pi}) \|_0$

$$I(J_{\hat{\pi}}) = I(J_{\pi}) \cdot P_{\rho}$$

The latent→action bipartite graph is identified up to a permutation ρ

Thm. 2 Block identification

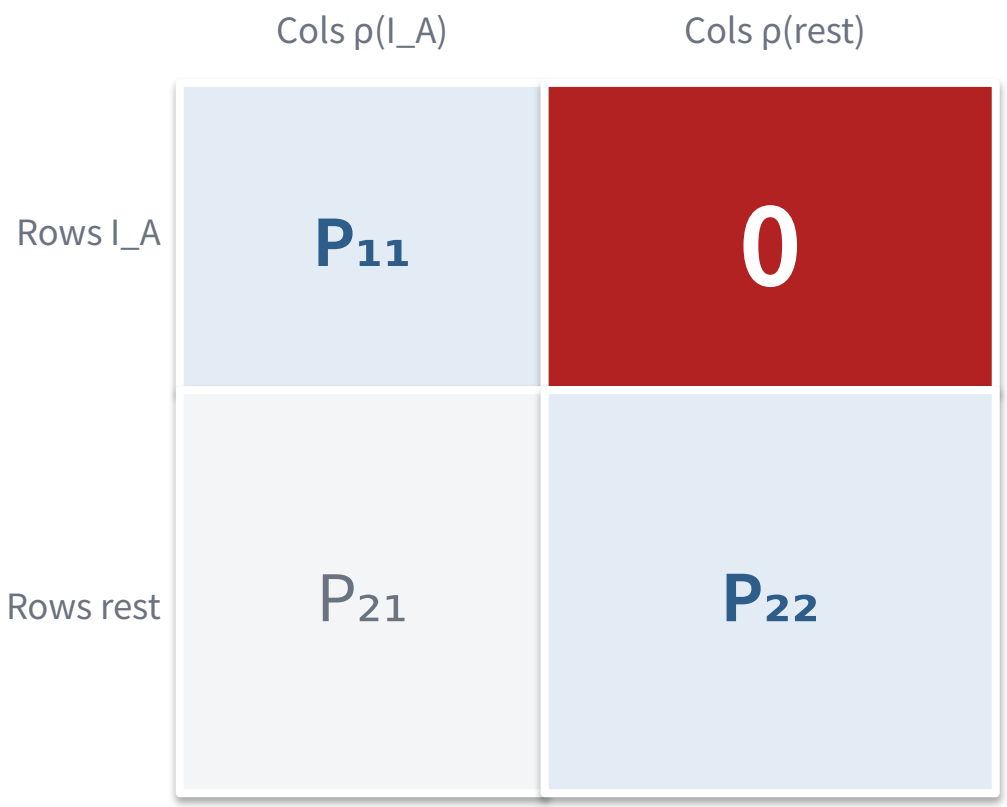
Same conditions

$$C_t = \hat{s}_{t,\rho(I_A)} = h(s_{t,I_A})$$

The compact action state is an invertible re-encoding of the true action-relevant block, separated from nuisances

Key selling point: the result is functional (C_t depends only on and determines s_IA) and holds however strongly action-relevant and nuisance variables are correlated.

Key step: support alignment zeroes blocks of the reparam. Jacobian



$P = J_{\phi^{-1}}$ (reparameterization between true s and estimate ŝ)

Top-right zero block ⇒ ŝ_ρ(I_A) = h(s_I_A), h invertible

① **Chain rule** $J_{\hat{\pi}} = J_{\pi} \cdot P$ links the two models

② **Span condition** each action dim's Jacobian rows span its true parents over several states, so reparameterization cannot erase true dependencies

③ **Common permutation** $\det P \neq 0 \Rightarrow$ the Leibniz expansion has a ρ with $\rho(I_i) \subseteq \hat{I}_i$ (shared across all action dims)

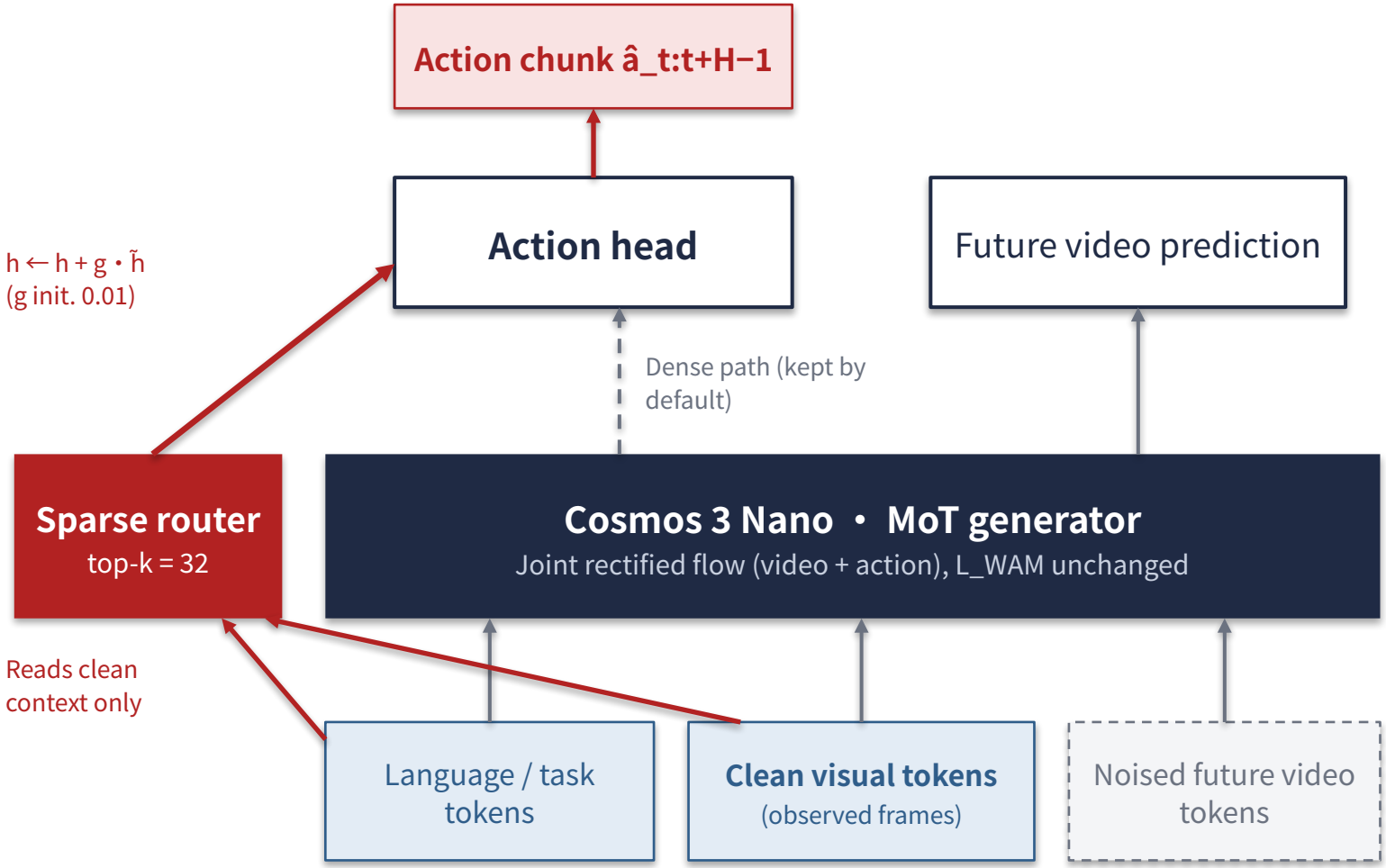
④ **Sparsity equality** total support ⩽ true ⇒ inclusion becomes equality ⇒ P block lower-triangular ⇒ block invertible

Cost of the assumptions

- The span condition is not free: e.g., $\pi_i = g(s_1+s_2)$ has collinear Jacobian rows and fails it
- Only block-level identification; latents within a block may stay mixed (acknowledged limitation)

Method

CWAM: same WAM loss + a sparse router beside the action head



Routing weights

$w_{ij} = \mathrm{softmax}_j(q_i^\top k_j/\sqrt{d_r} + b_{ij})$

b_{ij}: position / modality / chunk-progress bias

Sparsity surrogate loss (token-level approx. of Eq. 3)

$L_{sparse} = \lambda \cdot E_i[1 - \Sigma_{top\text{-}k} w_{ij}]$

Encourages attention mass on the top-k tokens

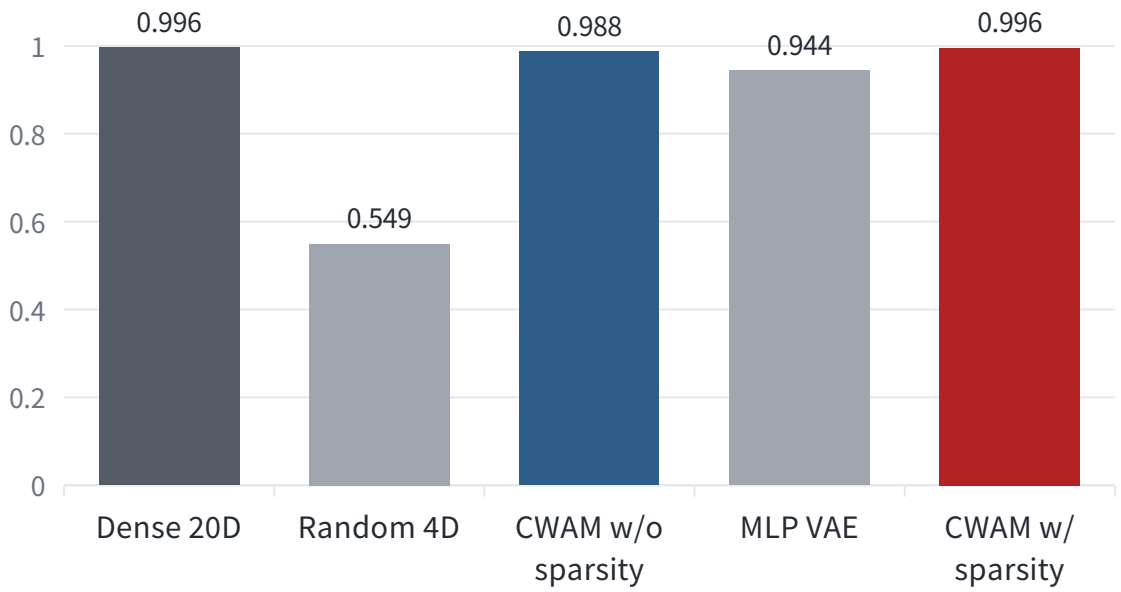
$L_{total} = L_{WAM} + L_{sparse}$

- $d_r = 512$, 8 heads, $k = 32$
- $\lambda = 0.01$, enabled at step 1k, linear ramp over 4k steps
- C_t = tokens in any action query's top-k

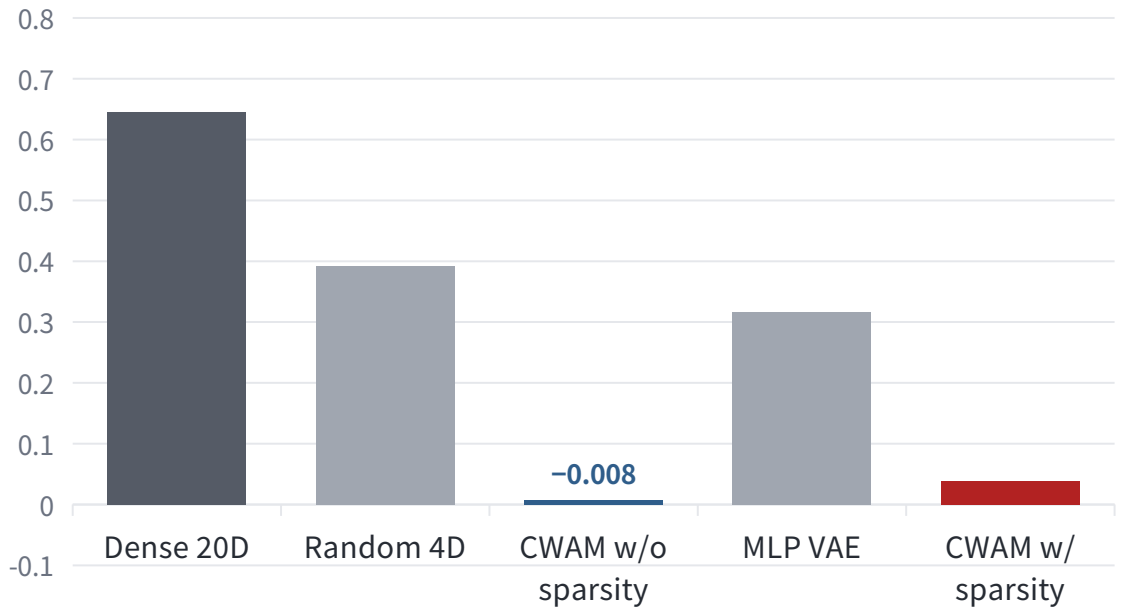
Experiment 1 • Synthetic data

Synthetic: sparsity-selected 4 dims hold almost only action-relevant info

Relevant R^2 ↑ (decodability of the action-relevant block)



Δ_nuis ↓ (excess nuisance decoding)



≈ 0

With sparsity, Δ_nuis = 0.039 vs. 0.646 for Dense

1.70×

Keeping only C_t raises action MSE just 1.70× over full state (random 4D: 48.1×)

Stable

As hidden environments E grow from 1 to 5, sparse Δ_nuis stays ≈ 0; Dense stays at 0.64–0.68

Closed-loop success up on 2 sim benchmarks + real stacking

RoboCasa365

0.24 → **0.43**

17 tasks × 100 episodes, macro avg.; 13/17 tasks improve

LIBERO

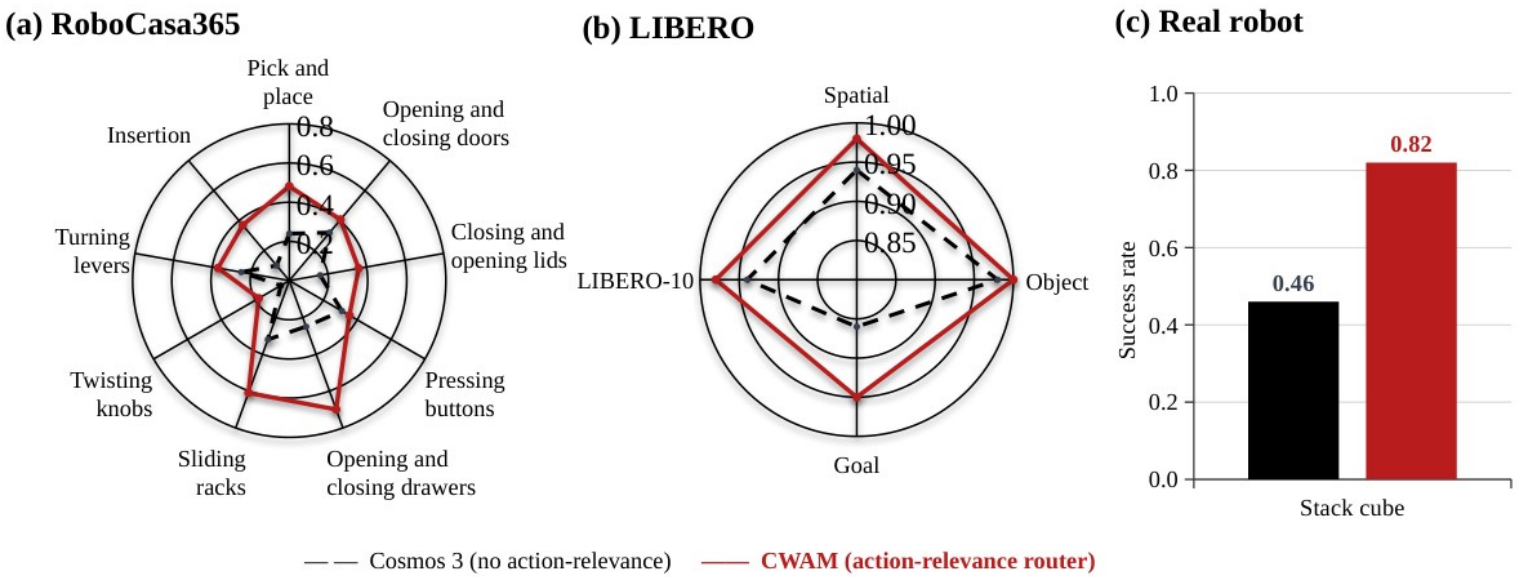
4 / 4 suites improve

40 tasks × 100 episodes; every suite higher, Goal gains most

Real Franka block stacking

0.46 → **0.82**

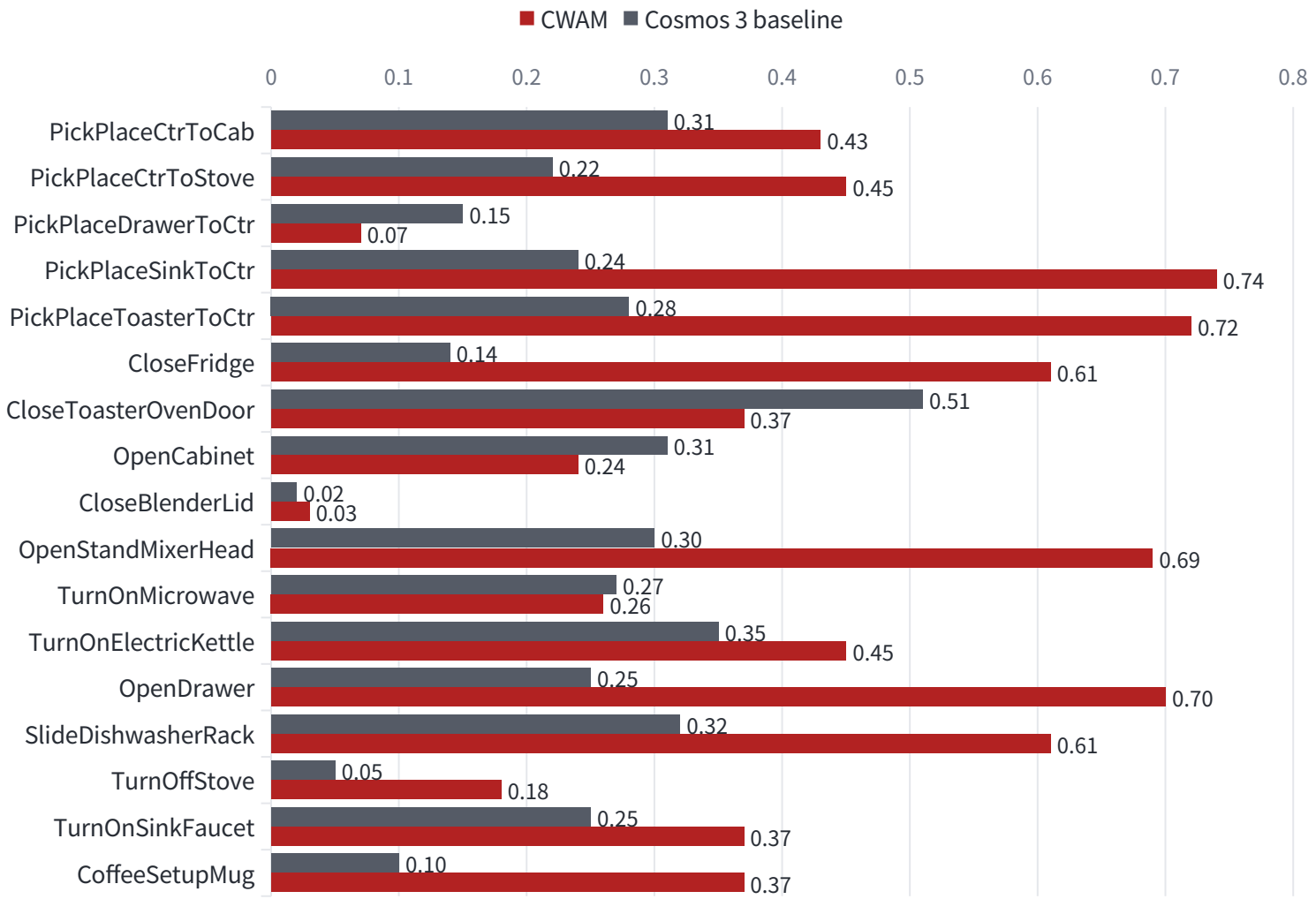
50 trials / policy



Original Fig. 2: (a) RoboCasa365 mean success per skill family (b) LIBERO per suite (c) real robot; black dashed = Cosmos 3 baseline, red = CWAM

Experiment 2 • Per task

RoboCasa365: 13/17 tasks improve, mainly pick-place & open/close



Macro avg.

0.24 → 0.43

Largest gains

SinkToCtr 0.24 → 0.74
ToasterToCtr 0.28 → 0.72
OpenDrawer 0.25 → 0.70
OpenStandMixerHead 0.30 → 0.69
CloseFridge 0.14 → 0.61

Declined or flat

DrawerToCtr 0.15 → 0.07
CloseToasterOvenDoor 0.51 → 0.37
OpenCabinet 0.31 → 0.24
TurnOnMicrowave 0.27 → 0.26

Routing hits manipulated objects & goals; those tokens suffice

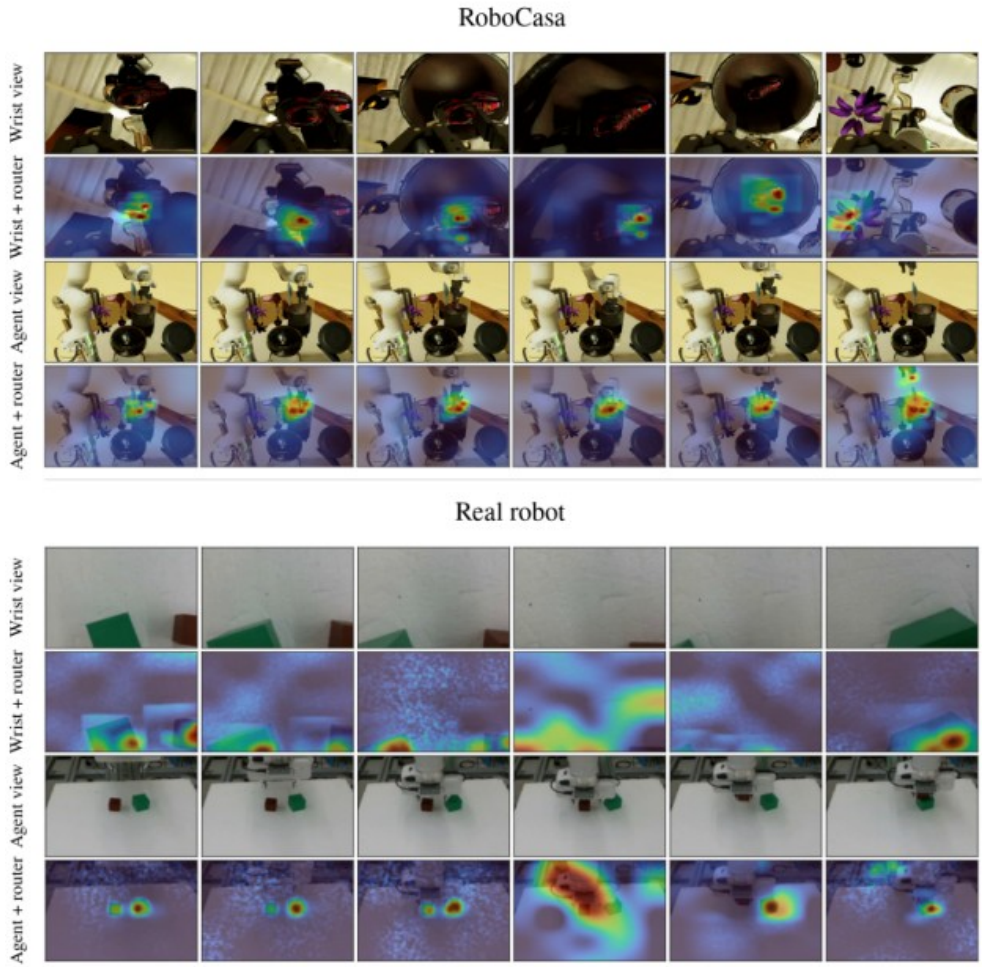
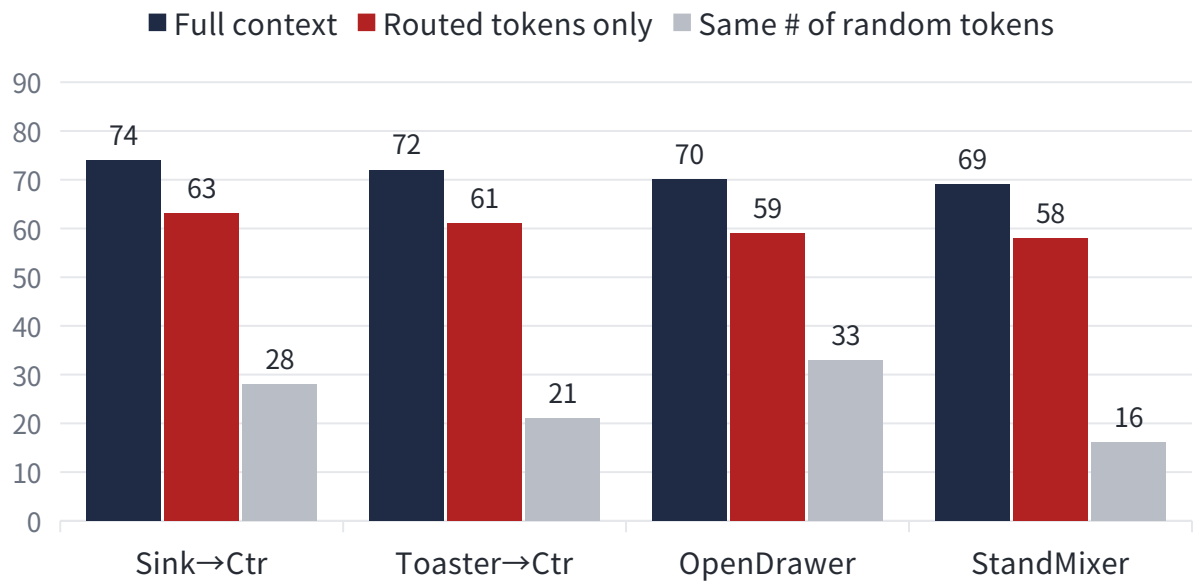


Fig. 3: top RoboCasa365, bottom real (red→green block); warm = top-k routing mass

Fixed 50k ckpt, no retrain; vary action queries' visual tokens



71.3%
Full context

60.3%
Routed tokens only

24.5%
Same # random

Mean success over 4 tasks (%). Only CWAM's best 4 tasks evaluated; in a few frames routing spreads to the arm and walls.

Assessment

Sober view: solid theory, large leap to practice, evidence gaps

Strengths

Clear problem framing “ prediction needs the whole scene, action only part” is formalized as an over-coverage lower bound + sparsity equality

Weak identification conditions no interventions, temporal dependence or Markovianity; correlations allowed

Plug-and-play post-training module, world-modeling objective unchanged; +36 points on the real robot

Interpretable routing heatmaps and token-intervention experiments corroborate each other

Concerns

Theory ≠ implementation theorems concern ℓ_0 support over latent coordinates and need the true support size; the implementation uses an attention top-k surrogate and keeps a dense path

Is sparsity necessary? in synthetic tests “no sparsity” gives Δ_nuis = −0.008, no worse than sparse; top-4 uses the true |I_A|

Missing key ablation no “router without L_sparse” control, so gains from added parameters can't be ruled out

Weak statistics no multiple seeds / confidence intervals for robot results; one real-robot task; RoboCasa baseline only 0.24

Table 2 too tidy C-only drops exactly 11 points on all 4 tasks, and only the best 4 tasks are tested

Takeaways

1. **Split prediction & action**
   WAM latents must describe the whole scene, but actions should come only from the causal block; fitting demos alone causes over-coverage.

2. **Sparsity suffices**
   Support sparsity on latent→action dependencies: structure identified up to permutation; the compact action state is an invertible function of the true action block.

3. **Works in practice**
   Cosmos 3 + sparse routing: RoboCasa365 0.24→0.43, gains on all LIBERO suites, real robot 0.46→0.82.

4. **Evidence still needed**
   Gap between token-level surrogate and theory, a router-without-sparsity control, multi-seed statistics.